

OBTENCIÓN Y CREACIÓN DEL DATASET

DATASET 1: AudioSet

Es la referencia estándar de clasificación de audio, con millones de clips de 10 segundos extraídos de YouTube. Extraer audios de AudioSet no se puede realizar de forma directa, sino que requiere de un script ya que la base de datos no contiene archivos de sonido directos, sino un enorme archivo CSV con enlaces a vídeos de YouTube y marcas de tiempo temporales. Cabe destacar que algunos vídeos de YouTube de este dataset no están disponibles por una serie de razones, como por ejemplo que el autor lo haya borrado.

- Requisitos
 - pip install pandas yt-dlp
 - FFmpeg (motor de procesamiento de audio que yt-dlp necesita para recortar los clips y extraer el audio sin descargar los vídeos enteros) (winget install ffmpeg)
- Paso 1: Descarga metadatos (<https://research.google.com/audioset/download.html>)
 - descarga del archivo *balanced_train_segments.csv* (contiene una mezcla equilibrada de clases, ideal para entrenar modelos de inteligencia artificial sin sesgos masivos).
 - descarga del archivo *eval_segments.csv* (para validar el modelo más adelante)
- Paso 2: Ontology ID (<https://github.com/audioset/ontology/blob/master/ontology.json>)
 - **Shout:** /m/07p6fty
 - **Yell:** /m/07sr1lc
 - **Screaming:** /m/03qc9zr
 - **Crying, sobbing:** /m/0463cq4
 - **Baby cry, infant cry:** /t/dd00002
 - **Wheeze:** /m/07mzm6
 - **Snoring:** /m/01d3sd
 - **Cough:** /m/01b_21
 - **Sneeze:** /m/01hsr_
 - **Thump, thud:** /m/07qnq_y
 - **Bang:** /m/07pws3f
 - **Silence:** /m/028v0c
 - **Speech:** /m/09x0r
 - **Whispering:** /m/02rtxlg
 - **Laughter:** /m/01j3sz
 - **Breathing:** /m/0lyf6
 - **Walk, footsteps:** /m/07pbtc8
 - **Burping, eructation:** /m/03q5_w
 - **Chatter:** /m/07rkbfh
 - **Thunderstorm:** /m/0jb2l
 - **Water:** /m/0838f
 - **Vehicle horn, car horn, honking:** /m/0912c9
 - **Car passing by:** /t/dd00134
 - **Emergency vehicle:** /m/03j1ly
 - **Traffic noise, roadway noise:** /m/0btp2
 - **Door:** /m/02dgv
 - **Alarm clock:** /m/046dlr

- **Inside, small room:** /t/dd00125
- Paso 3: Script de filtrado y descarga
 - Fichero *audioSet_extractor.py*
- Resultados: 1378 muestras para el conjunto train y 1444 muestras para el conjunto eval.

DATASET 2: ESC-50

A diferencia de AudioSet, ESC-50 es mucho más ligero y directo, ideal para obtener muestras muy limpias de los episodios de tos, ronquidos o lloros que el sistema necesita clasificar. La versión oficial agrupa sus 2000 audios de 5s en una única carpeta, así que usaremos un pequeño script para extraer los que necesitamos basándonos en su código interno. Contiene 2000 audios en formato .wav (5s, 44.1kHz, mono) con el siguiente formato en cuanto a los nombres de los archivos: {FOLD}-{CLIP_ID}-{TAKE}-{TARGET}.wav

- {FOLD} - index of the cross-validation fold
- {CLIP_ID} - ID of the original Freesound clip,
- {TAKE} - letter disambiguating between different fragments from the same Freesound clip,
- {TARGET} - class in numeric format [0, 49].
 - Paso 1: Descarga del dataset y metadatos (<https://github.com/karolpiczak/esc-50>)
 - Metadatos: *esc50.csv*
 - Paso 2: Ontology ID
 - **Coughing:** 24
 - **Crying baby:** 20
 - **Sneezing:** 21
 - **Snoring:** 28
 - **Rain:** 10
 - **Toilet flush:** 18
 - **Thunderstorm:** 19
 - **Breathing:** 23
 - **Footsteps:** 25
 - **Laughing:** 26
 - **Clock alarm:** 37
 - **Siren:** 42
 - Paso 3: Script de filtrado
 - Fichero *esc50_extractor.py*
 - Resultados: 480 muestras.

DATASET 3: FSD50K

FSD50K es un dataset masivo alojado en la plataforma científica Zenodo e impulsado por la Universitat Pompeu Fabra. Su estructura requiere cruzar los nombres numéricos de los archivos de audio con un archivo CSV de metadatos para aislar las incidencias a clasificar.

- Paso 1: Descarga del dataset y metadatos (<https://zenodo.org/records/4060432>)
 - Metadatos: *FSD50K.ground_truth.zip*
- Paso 2: Ontology ID
 - **Burping and eructation:** /m/03q5_w
 - **Cough:** /m/01b_21

- **Crying and sobbing:** /m/0463cq4
- **Respiratory sounds:** /m/09hlz4
- **Screaming:** /m/03qc9zr
- **Shout:** /m/07p6fty
- **Sneeze:** /m/01hsr_
- **Thump and thud:** /m/07qnq_y
- **Yell:** /m/07sr1lc
- **Alarm:** /m/07pp_mv
- **Breathing:** /m/0lyf6
- **Car passing by:** /t/dd00134
- **Chatter:** /m/07rkbfh
- **Conversation:** /m/01h8n0
- **Door:** /m/02dgv
- **Human voice:** /m/09l8g
- **Laughter:** /m/01j3sz
- **Rain:** /m/06mb1
- **Siren:** /m/03kmc9
- **Speech:** /m/09x0r
- **Thunderstorm:** /m/0jb2l
- **Traffic and roadway noise:** /m/0btp2
- **Vehicle horn and car horn and honking:** /m/0912c9
- **Walk and footsteps:** /m/07pbtc8
- **Water:** /m/0838f
- Paso 3: Script de filtrado
 - Fichero `fsd50k_extractor.py`
- Resultados: 3950 muestras del conjunto eval y 9692 muestras del conjunto dev.

DATASET 4: COUGHVID

El dataset COUGHVID es un dataset de salud masivo creado durante la pandemia, repleto de muestras de toses aportadas por todo el mundo. Es la mejor fuente para entrenar el modelo en la detección de la incidencia de *tos persistente*. Como la base de datos contiene más de 20000 grabaciones y muchas tienen ruido o silencios, filtraremos estrictamente aquellas que el propio dataset certifica que contienen tos clara. El archivo CSV contiene una columna llamada *cough_detected*. Es un valor del 0 al 1 que indica la probabilidad de que el audio contenga tos real. Filtraremos solo aquellos con un valor superior o igual a 0.95 (95% de confianza).

- Paso 1: Descarga del dataset y metadatos (<https://zenodo.org/records/4048312>)
 - Metadatos: *metadata_compiled.csv*
- Paso 2: Criterios de selección
 - **Cough detected:** ≥ 0.95
 - **quality_1 o quality_2 o quality_3:** {ok, good} o vacía
 - **severity_1 o severity_2 o severity_3:** {severe} o vacía
- Paso 3: Script de filtrado
 - Fichero *coughvid_extractor.py*
- Resultados: 5902 muestras.

TOTAL: 22846 muestras

FORMATO DE LOS AUDIOS

Se usará el Formato 16 kHz mono, ya que es el estándar TinyML. La frecuencia de muestreo (16 kHz) significa que el audio captura 16000 puntos de datos por segundo. Según el teorema de Nyquist, esto permite registrar frecuencias de hasta 8 kHz. Como la voz humana, los ronquidos y los golpes concentran casi toda su información acústica por debajo de los 4 kHz, no perdemos información; usar 44.1 kHz (calidad CD) solo añadiría frecuencias agudas inútiles y triplicaría el peso del archivo, colapsando la memoria RAM del microcontrolador durante la generación del espectrograma. El formato mono indica que el audio solo tiene 1 canal. Los micrófonos estéreo generan dos pistas de audio simultáneas (izquierda y derecha), pero el modelo no necesita saber de qué lado del dispositivo proviene el sonido, solo si ha ocurrido y cómo ha ocurrido. El formato mono divide el procesamiento matemático a la mitad.

Los scripts que obtienen audios de AudioSet y COUGHVID ya integran FFmpeg para forzar el formato `-ar 16000 -ac 1` durante la extracción. Sin embargo, ESC-50 y FSD50K realizan una copia directa del .wav original usando `shutil.copy2`. Para estandarizar estos dos últimos y unificar todo el dataset, se ejecuta el script *UNIFICADOR.py*.

De hecho, se ejecuta también el script que aplica la función FFmpeg a los audios de los dos primeros datasets por los motivos que siguen:

- Sin pérdida de calidad: Como los archivos .wav utilizan formato PCM (audio sin compresión), recodificar un audio que ya está a 16 kHz mono bajo esos mismos parámetros es un proceso transparente que no degrada el sonido, a diferencia de lo que ocurriría con formatos comprimidos como MP3.
- Limpieza de metadatos: FFmpeg reescribirá desde cero la cabecera técnica del archivo WAV. Esto es muy beneficioso para unificar el dataset, ya que elimina metadatos residuales, etiquetas extrañas o codificaciones anómalas que pudieran venir de los datasets originales y que a veces generan errores de lectura al subir los datos a Edge Impulse.
- Impacto mínimo en el sistema: El único efecto real es que el procesador invertirá unas milésimas de segundo extra en leer y reescribir un archivo que ya tenía el formato correcto, un tiempo totalmente despreciable.

PREPROCESSING?

Aplicar técnicas de aumento de datos (Data Augmentation) es, sin duda, el "arma secreta" cuando desarrollamos modelos de TinyML (Machine Learning en microcontroladores). Como el **AI Night Guardian** ejecutará la inferencia local en un Arduino Uno Q, el modelo debe ser pequeño pero extremadamente generalista.

Si entrenamos la inteligencia artificial solo con audios limpios, sufrirá de *sobreajuste* (overfitting): memorizará los ruidos exactos del dataset y fallará en el mundo real. Para evitarlo, multiplicaremos artificialmente nuestro dataset aplicando las siguientes transformaciones matemáticas a las muestras originales.

Técnicas Clave de Aumento de Datos Acústicos

Aquí tienes las estrategias más efectivas para asegurar que vuestro sistema sea robusto y detecte las incidencias con precisión sin generar falsas alarmas que saturen la aplicación móvil del personal.

1. Inyección de Ruido de Fondo (Background Noise Addition)

- **En qué consiste:** Mezclar los audios limpios de vuestras clases objetivo con ruido ambiental.
- **Aplicación al proyecto:** Añadid el zumbido de un sistema de ventilación, el ruido de la calle de fondo o estática a las grabaciones de roncs o plors.
- **Beneficio:** Evita que el modelo se confunda si un paciente empieza a toser mientras el aire acondicionado está encendido. Enseña a la red neuronal a "separar" el sonido importante del ruido cotidiano antes de que la finestra d'àudio s'esborra immediata i irreversiblement.

2. Variación de Tono (Pitch Shifting)

- **En qué consiste:** Subir o bajar el tono del audio sin cambiar su duración.
- **Aplicación al proyecto:** La voz humana varía enormemente. Los crits demanant ajuda de un hombre con voz grave suenan muy distintos a los de una mujer con voz aguda. Lo mismo ocurre con los plors de nens petits.
- **Beneficio:** Con una sola grabación de un grito de auxilio, podéis generar cinco variantes simulando pacientes con diferentes tamaños de cuerdas vocales.

3. Ajuste de Ganancia y Distancia (Random Gain & Panning)

- **En qué consiste:** Alterar el volumen general de la grabación aleatoriamente.
- **Aplicación al proyecto:** Un cop o caiguda no sonará con la misma intensidad si ocurre a medio metro del mòdul micròfon I2S que si ocurre en el baño al fondo de la habitación.
- **Beneficio:** Entrena al modelo para reconocer que un sonido tenue pero con un patrón acústic identificable de impacto sigue siendo una caída, y no debe ignorarlo por sonar "lejos".

4. Estiramiento Temporal (Time Stretching)

- **En qué consiste:** Acelerar o ralentizar el audio sin alterar su tono.
- **Aplicación al proyecto:** Una tos persistent puede tener un ritmo rápido y entrecortado, o ser lenta y espaciada.

- **Beneficio:** Expande la variedad de cadencias rítmicas que el modelo puede comprender.

5. Enmascaramiento de Espectrograma (SpecAugment)

- **En qué consiste:** Una vez convertido el audio en una imagen visual de sus frecuencias (espectrograma), se tapan aleatoriamente bloques de tiempo o de frecuencia con "cuadrados negros" durante el entrenamiento.
- **Aplicación al proyecto:** Es una técnica muy avanzada pero fácil de implementar. Obliga a la red neuronal a adivinar el sonido basándose en pistas incompletas.
- **Beneficio:** Mejora radicalmente la robustez. Si el entorno acústico de la habitación distorsiona una parte de la frecuencia del sonido, el modelo aún será capaz de identificar qué ha sucedido.

Herramientas Recomendadas

Para implementar todo esto de forma programática y automática (por ejemplo, en Python antes de subir el modelo al hardware), os recomiendo utilizar librerías de código abierto muy consolidadas:

- **Audiomentations:** Una librería excelente diseñada específicamente para crear *pipelines* de aumento de audio.
- **Librosa:** El estándar de facto para análisis de audio en Python, perfecta para hacer *Pitch Shifting* y *Time Stretching*.

Aplicando estas variaciones, un dataset inicial de 5.000 audios puede convertirse fácilmente en 25.000 muestras altamente representativas del caos acústico de la vida real, manteniendo el compromiso de no gravar ni emmagatzemar converses de los residentes de manera permanente.

Alimentar una red neuronal directamente con el audio crudo (las ondas de sonido punto por punto) es computacionalmente inviable para un modelo que ejecuta la inferencia localmente en un microcontrolador, incluso contando con los 4GB o más del Arduino Uno Q. Para que el sistema sea rápido, eficiente y cumpla con que la señal acústica se capte en formato streaming y la ventana de audio se borre inmediata e irreversiblemente, necesitamos extraer las "características" (features) del sonido.

Aquí es donde entran en juego las matemáticas. En lugar de procesar todo el audio, transformaremos el sonido en representaciones visuales o numéricas mucho más ligeras que capturan su esencia.

Técnicas de Extracción para TinyML

Existen dos estándares principales en la industria para analizar audio en dispositivos de bajos recursos. Ambos se basan en la **Escala Mel**, que imita matemáticamente cómo el oído humano percibe las frecuencias (somos más sensibles a los cambios en tonos bajos que en tonos altos).

1. Espectrogramas Mel (Mel Spectrograms)

- **Qué es:** Convierte el fragmento de audio en una imagen 2D (un mapa de calor) donde el eje X es el tiempo, el eje Y es la frecuencia (en escala Mel) y los colores representan la intensidad (volumen).
- **Ventaja para el proyecto:** Es ideal para identificar eventos con un patrón acústico identificable muy visual, como los impactos secos de golpes o caídas.
- **Consideración de Hardware:** Requiere un poco más de memoria RAM para almacenar la "imagen" antes de la inferencia, pero los 4GB de vuestra placa lo manejarán sin ningún problema.

2. Coeficientes MFCC (Mel-Frequency Cepstral Coefficients)

- **Qué es:** Es una versión aún más comprimida del Espectrograma Mel. Mediante una transformación matemática adicional (Transformada Discreta del Coseno), se extraen solo unos pocos valores (generalmente 13 o 40 coeficientes) que describen la "forma" general del sonido.
- **Ventaja para el proyecto:** Es el estándar de oro para el reconocimiento de voz. Será la técnica más precisa para que el modelo detecte eventos vocales como ronquidos, gritos pidiendo ayuda, llantos o una tos persistente.
- **Consideración de Hardware:** Son extremadamente ligeros. Minimizan la carga de procesamiento, asegurando que el dispositivo actúe legalmente como un sensor de emergencias automatizado de respuesta ultrarrápida.

El "Pipeline" dentro del Arduino Uno Q

Para que os hagáis una idea de cómo se programaría este flujo dentro del microcontrolador (en C++ o mediante un framework de TinyML), el proceso en tiempo real sería el siguiente:

1. **Captura:** El módulo micrófono I2S graba una ventana de audio de, por ejemplo, 1 o 2 segundos.
2. **Enventanado (Windowing):** El software divide ese segundo en fragmentos minúsculos de unos pocos milisegundos.
3. **FFT (Transformada Rápida de Fourier):** Se calculan las frecuencias presentes en esos fragmentos.
4. **Filtros Mel & MFCC:** Se aplican las transformaciones matemáticas para generar los coeficientes.
5. **Borrado e Inferencia:** La ventana de audio original se borra inmediata e irreversiblemente. Los coeficientes (no el audio) pasan por el modelo de IA.
6. **Notificación:** Si el resultado coincide con una anomalía, se registra el resultado de la clasificación en formato texto (por ejemplo: "Habitación 12 03:17 Golpe detectado") y se envía a la aplicación móvil.

Ambos enfoques son válidos e incluso se pueden combinar dependiendo de la arquitectura de la red neuronal.

Extracción de Características Integrada: Edge Impulse tiene bloques de procesamiento de señales (DSP) nativos tanto para **Espectrogramas Mel** como para **MFCC**. No tendréis que programar las complejas matemáticas de conversión de audio en C++ desde cero; la plataforma lo hace por vosotros.

Gestión Visual del Dataset: Os permitirá subir todo ese dataset masivo que hemos diseñado (combinando fuentes públicas y grabaciones con el micrófono I2S), etiquetar las incidencias (roncs, tos, cops, caigudes, crits, plors) y aplicar *Data Augmentation* (añadir ruido) con un par de clics.

Despliegue Directo al Hardware: Una vez entrenada la red neuronal, Edge Impulse empaqueta todo el *pipeline* (procesamiento de audio + modelo de IA) y lo exporta como una **librería de Arduino estándar (.zip)**, lista para ser compilada e instalada en vuestro Arduino Uno Q.